

1. Delete a node from a binary search tree.

```
2. class Solution {
3.     TreeNode findSuccessor(TreeNode root) {
4.         while (root.left != null) {
5.             root = root.left;
6.         }
7.         return root;
8.     }
9.
10.    public TreeNode deleteNode(TreeNode root, int key) {
11.        if (root == null) return null;
12.
13.        if (key > root.val) {
14.            root.right = deleteNode(root.right, key);
15.        } else if (key < root.val) {
16.            root.left = deleteNode(root.left, key);
17.        } else {
18.            if (root.left == null) return root.right;
19.            if (root.right == null) return root.left;
20.
21.            TreeNode temp = findSuccessor(root.right);
22.            root.val = temp.val;
23.            root.right = deleteNode(root.right, temp.val);
24.        }
25.
26.        return root;
27.    }
28. }
```

2. Check if a binary tree is a binary search tree.

```
class Solution {
    boolean check(TreeNode root, long min, long max) {
        if (root == null) return true;

        if (root.val <= min || root.val >= max) {
            return false;
        }

        return check(root.left, min, root.val) &&
            check(root.right, root.val, max);
    }

    boolean isBST(TreeNode root) {
        return check(root, Long.MIN_VALUE, Long.MAX_VALUE);
    }
}
```

3. find kth smallest and kth largest elements in bst

```

1  class Solution {
2      List<Integer> result = new ArrayList<>();
3      void inorder(TreeNode root) {
4          if (root == null)
5              return;
6          inorder(root.left);
7          result.add(root.val);
8          inorder(root.right);
9      }
10     public int[] kthSmallestAndLargest(TreeNode root, int k) {
11         inorder(root);
12         return new int[] {
13             result.get(k - 1),
14             result.get(result.size() - k)
15         };
16     }
17 }

```

4. Find whether the given BST is balanced or not.

```

class Solution {
    boolean flag = true;
    int height(TreeNode root){
        if(root == null) return 0;
        int lh = height(root.left);
        int rh = height(root.right);
        if(Math.abs(lh-rh)>1) flag = false;
        return 1+Math.max(lh,rh);
    }
    public boolean isBalanced(TreeNode root) {
        height(root);
        return flag;
    }
}

```